



Traitement d'images

Chapitre 6 – La Compression d'images

Wided SOUID MILED

Plan du cours

- Généralités sur la compression de données
- La compression sans pertes
 - Codage RLE
 - Codage de Huffman
- La compression avec pertes
 - Transformée en Cosinus Discrète (DCT)
 - La compression JPEG

Motivations

POURQUOI COMPRESSER ?

Combien de bits pour coder une seconde de vidéo Full HD ?

Caractéristiques d'une vidéo HD :

- 1920 x 1080 pixels
- 50 Hz
- Trois canaux (RGB) codés sur 8 bits = 1 octet chacun

$$1920 \times 1080 \times 50 \times 3 \times 1 = 311\,040\,000 \text{ octets } 311 \text{ Mo}$$

Un DVD (4,7 Go) ne peut donc stocker que 15 s...

Motivations

LA COMPRESSION PERMET DE :

- Transformer les données multimédia en une représentation **plus compacte** occupant moins d'espace qu'avant compression.
- Pallier aux insuffisances des capacités de **stockage** en mémoire et des vitesses de **transmission** sur les réseaux

Motivations

POURQUOI EST-IL POSSIBLE DE COMPRIMER ?

- **Redondance statistique** des données
 - homogénéité des images
 - similitude entre images successives
- **Redondance psycho visuelle**
 - sensibilité aux basses fréquences
- autres limites du **système visuel humain**
- Une méthode de compression (ou codage) doit exploiter au maximum la redondance des données

Les méthodes de compression



Méthodes sans perte (lossless)

- Reconstruction parfaite ($X=X'$)
- Basés sur la redondance statistique
- Faible taux de compression de 5 à 6

Méthodes avec perte (lossy)

- Image reconstruite $\neq$ image originale ($X \neq X'$)
- Plusieurs applications n'exigent pas une compression sans perte
- Redondance psychovisuelle : "visually lossless"
- Rapport de compression élevé (supérieur à 100)
- Compromis entre qualité et compression

Critères de performances

Les différents algorithmes de compression sont basés sur les critères suivants:

- **Le taux de compression**
c'est le rapport du volume de données original sur la taille du volume de données compressé
- **La qualité de compression**
sans ou avec pertes (avec le % de perte)
La distorsion du signal comprimé
- **La complexité du système**: coût calcul, mémoire requise, vitesse..

→ Trouver un compromis entre la performance de la compression (taux et qualité) et les contraintes logicielles (ex : temps-réel) et matérielles

Critères de performances

- **Taux de compression:** $TC = \frac{\text{Taille de l'image avant compression}}{\text{Taille de l'image après compression}}$
- **Débit de codage** $R = \frac{\text{Nb Total de bits après compression}}{\text{Nb de pixels}} [bpp]$

Critères de performances

Les Critères objectifs de distorsion sont des fonctions mathématiques de l'image d'origine X et de l'image reconstruite $\hat{X}$ après compression

Erreur quadratique moyenne (MSE) :
$$D = \frac{1}{M \times N} \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} (X_{n,m} - \hat{X}_{n,m})^2$$

Rapport signal sur bruit PSNR:
$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{D} \right)$$

Les Critères subjectifs sont basés sur l'évaluation de la qualité des images faite par des humaines

- Difficulté de créer un bon modèle du SVH
- Mesure de qualité perceptuelles, SSIM
- Analyse expérimentale, analyse statistique des résultats
- Evaluations longues, difficiles et coûteuses

Compression sans pertes

La compression sans pertes

- Lorsque les données sont compressées et ensuite décompressées, aucune information **n'est perdue** ou modifiée.

critère objectif : minimiser le code binaire en évitant la distorsion de l'image (erreur nulle entre l'image originale et l'image compressée)

- Utile lorsqu'on veut garder une grande précision (domaine médical, archivage, etc.)
- **Algorithmes de compression sans pertes:**
 - Méthodes à base de redondances (RLE)
 - Méthodes statistiques (Huffman, ...)
 - Méthodes à base de dictionnaires (LZW, ...)

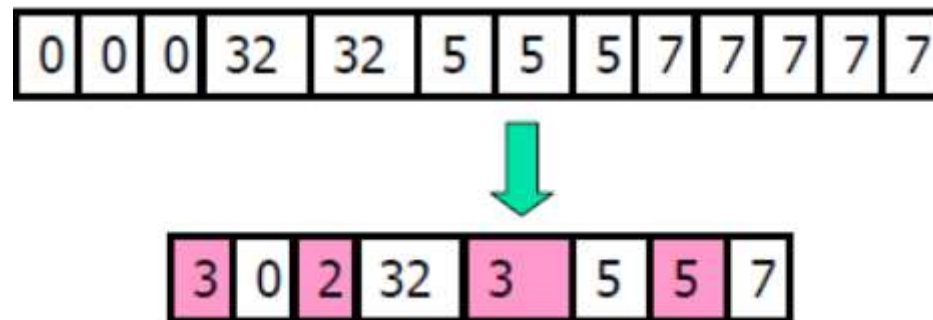
Codage RLE (Run Length Encoding)

Le **Run-length encoding** RLE ou *codage par plage* code la répétition de caractères identiques en donnant le nombre de répétitions et la valeur du caractère à répéter.

Principe:

- Recherche de séquences de données redondantes (plages de répétitions)
- La chaîne répétée est appelée un passage (run) et est codée avec deux bits: le premier représente le nombre de caractères dans le passage et est appelé **compteur (run count)** et le second représente la **valeur (run value)**
- Création d'une nouvelle séquence: (run count , run value)

Exemple:



Codage RLE (Run Length Encoding)

Application aux images

1) Application aux images monochrome

- Deux couleurs possibles : Noir (0) et Blanc (1)

Ex: 00000111101100000001111 ...

- On suppose que la couleur de départ est le noir. On compresse alors uniquement en indiquant le nombre de pixels Noir ou Blanc successifs

Ex : 5 4 1 2 7 4 0 ...



Et s'il y a plus de 255 fois le même pixel (codage 8 bits) ?

Ex : 1111000...00011
 ↔
 400x

Solution : insertion d'un 0 tous les 255 pixels.

Ex : 0 4 255 0 145 2 (d) = 00 04 FF 00 91 02 (h)

Codage RLE (Run Length Encoding)

2) Application aux images en niveaux de gris

Ex : Image possédant une profondeur de 8 bits :

... 12,12, 12, 12, 12, 35, 76, 112, 67, 87, 87, 87, 87 ...

codée en : 5 12 35 76 112 67 4 87



Comment distinguer ces nombres?

- Limiter l'image a 255 niveaux de gris (0 → 254), le 256^{ème} (255) servant de flag, précédant chaque octet correspondant a une longueur de chaine

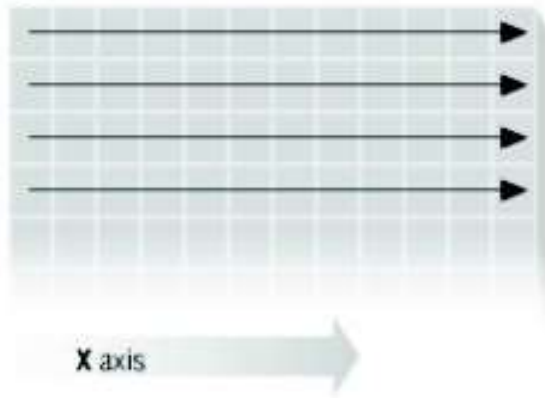
Ex : ... 12,12, 12, 12, 12, 35, 76, 112, 67, 87, 87, 87, 87 ...

devient ... 255, 5, 12, 35, 76, 112, 67, 255, 4, 87 ...

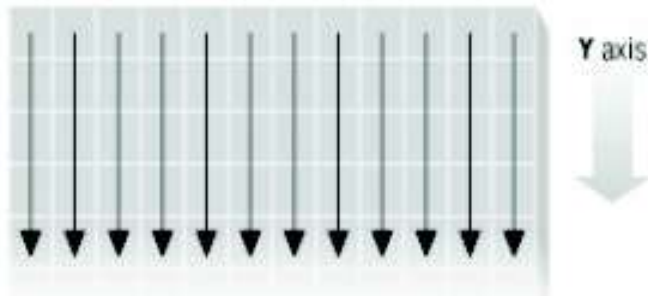
Codage RLE (Run Length Encoding)

Comment parcourir l'image?

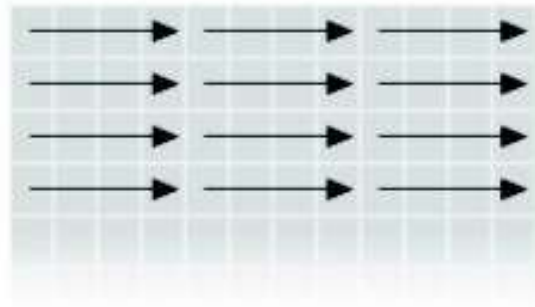
a Encoding along the X axis



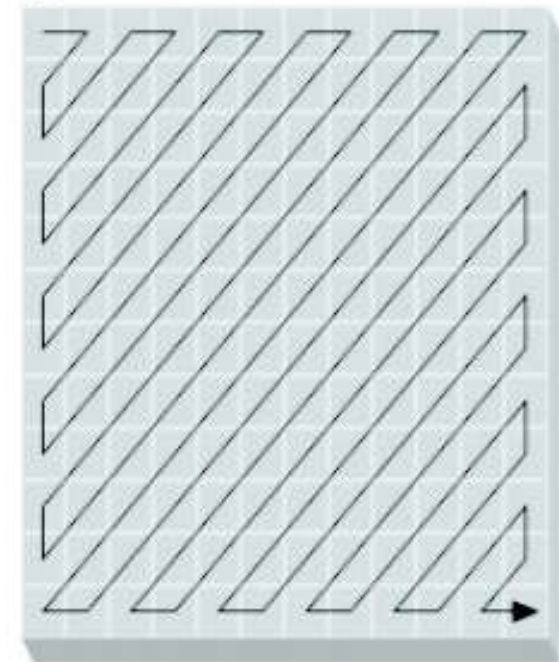
b Encoding along the Y axis



c Encoding (4x4 pixel) tiles



d Zig-zag encoding



Codage RLE (Run Length Encoding)

- RLE est utilisé par la plupart des formats de fichiers bitmaps tels que TIFF, BMP, etc.
- Permet de compresser n'importe quel type de donnée sans tenir compte de l'information qu'elle contient.

Avantages

- Algorithmes de compression et décompression très simples et rapides.

Limites

- Efficace seulement pour de nombreuses et longues plages constantes.
 - Cas des images de synthèse simples (possédant de nombreuses zones de régions homogènes) ;
 - peu adapté aux photos.
- Nécessite de fixer un maximum pour la longueur des plages (ex. 255).
- Ne permet pas d'atteindre de forts taux de compression

Codage de Huffman

Principe:

- Coder les valeurs des pixels avec un nombre de bits différent.
 - Code optimal à **longueur variable** (produisant la longueur moyenne la plus faible)
 - dit aussi **codage entropique**
- Plus une valeur apparaît fréquemment, plus le nombre de bits utilisés pour la coder est petit (*i.e.* plus son code est court).

Algorithme de Huffman

Phase 1 : Construction de l'arbre.

1. Trier les différentes valeurs par ordre décroissant de fréquence d'apparition
→ table de **poids**.
2. Fusionner les deux poids minimaux dans un arbre binaire et affecter leur somme à un nouveau noeud.
3. Réordonner la table de poids par poids décroissants.
4. Recommencer en 2. jusqu'à arriver à la racine de l'arbre.

Phase 2 : Construction du code à partir de l'arbre obtenu dans la phase 1.

À partir de la racine, attribuer des 0 aux branches de gauche et des 1 à droite

Codage de Huffman

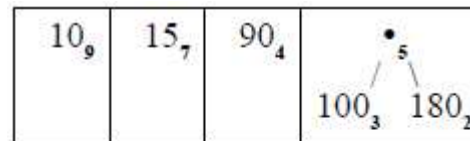
Exemple de Codage:

- Construction de l'arbre:

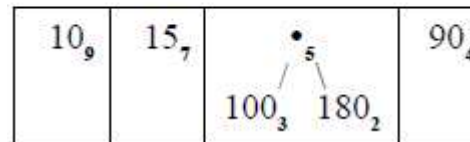
1. Table des poids

10_9	15_7	90_4	100_3	180_2
--------	--------	--------	---------	---------

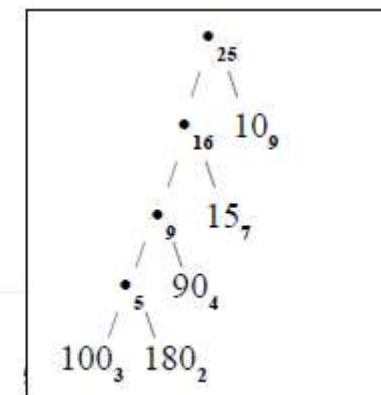
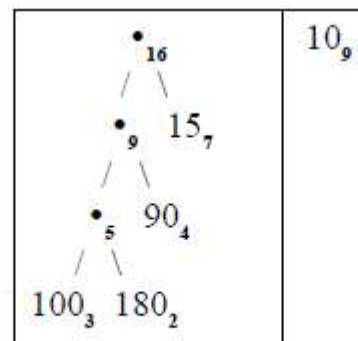
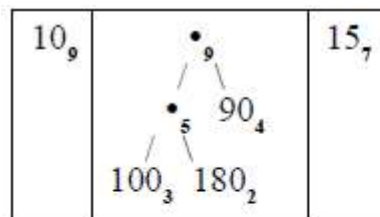
2. Fusion des poids minimaux



3. Réordonnancement



4. Itérations



10	15	15	15	15
10	90	100	100	15
10	90	180	100	15
10	90	180	90	15
10	10	10	10	10

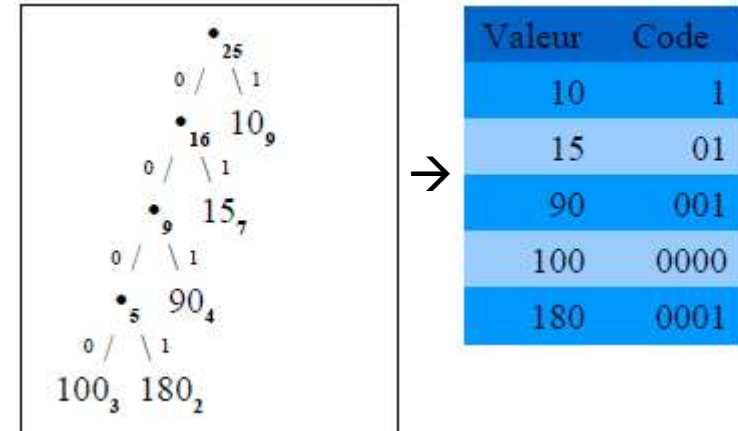
Codage de Huffman

Exemple de Codage:

- **Construction du code:**

1. Affectation des valeurs binaires aux branches gauches et droites
2. On obtient les mots de code pour chaque valeur

Image codée en ligne: 1 01 01 01 01 1 001 0000..
Soit 55 bits au lieu de $25 \times 8 = 200$ bits



Décodage

- Propriété du **préfixe** : aucun code n'est le préfixe d'un autre

→ code instantané et décodage non ambigu

- Le décodeur doit connaître la table de codage (entête) ;

Entrée	Action	Buffer	Émission
1	Identification de 10	vide	10
0	Bufferise et attend	0	rien
1	Identification de 15	vide	15
0	Bufferise et attend	0	rien
1	Identification de 15	vide	15
...	...	...	...

Codage de Huffman

- Surtout efficace pour des images ≤ 256 couleurs.
- Technique de codage très souvent utilisée dans d'autres méthodes de compression (ex: images TIFF, JPEG, ou MPEG, ...)
- Mais ...
- nécessite la connaissance préalable des probabilités d'apparition des symboles de source.
- Nécessité de transmettre l'arbre pour la décompression

Compression avec pertes

La compression avec pertes

- Les méthodes de **compression sans pertes** ne sont pas satisfaisantes pour des images en niveau de gris ou couleur, d'où le recours à des méthodes de **compression avec pertes**
- Les techniques de **compression avec pertes** sont basées sur le fait que les niveaux de gris des pixels voisins sont fortement corrélés. On parle alors de **redondance spatiale**.
- Suppression des informations les moins indispensables pour l'œil humain (Réduction de la quantité de données)
- Taux de compression plus élevé que la compression sans pertes.
- Plus le taux de compression est élevé, plus le niveau de pertes est important et plus la qualité de l'image compressée est dégradée

Premières méthodes intuitives

Sous échantillonnage : on ignore simplement certains pixels. Les effets sur l'image sont très visibles (grande perte de détails) : cette méthode simple n'est que très peu utilisée.



Originale



4 fois moins de pixels



16 fois moins de pixels

Premières méthodes intuitives

Quantification : diminuer les niveaux de gris de l'image.



Originale



32 niveaux de gris



16 niveaux de gris

Idée générale

- Transporter l'image dans un espace où l'information (l'intensité lumineuse) est concentrée en un petit nombre de points.
- Appliquer une transformation orthogonale à l'image (application linéaire) qui permet de:
 - réduire la redondance de l'image,
 - Séparation des données entre parties importantes et pas importantes
- Les parties importantes et moins importantes sont le plus souvent identifiées en travaillant sur les différentes fréquences constituant l'image (Les basses fréquences correspondent aux éléments importants d'une image, tandis que les hautes fréquences décrivent les détails d'une image).

Quelle transformation choisir?

Transformée en cosinus discrète DCT

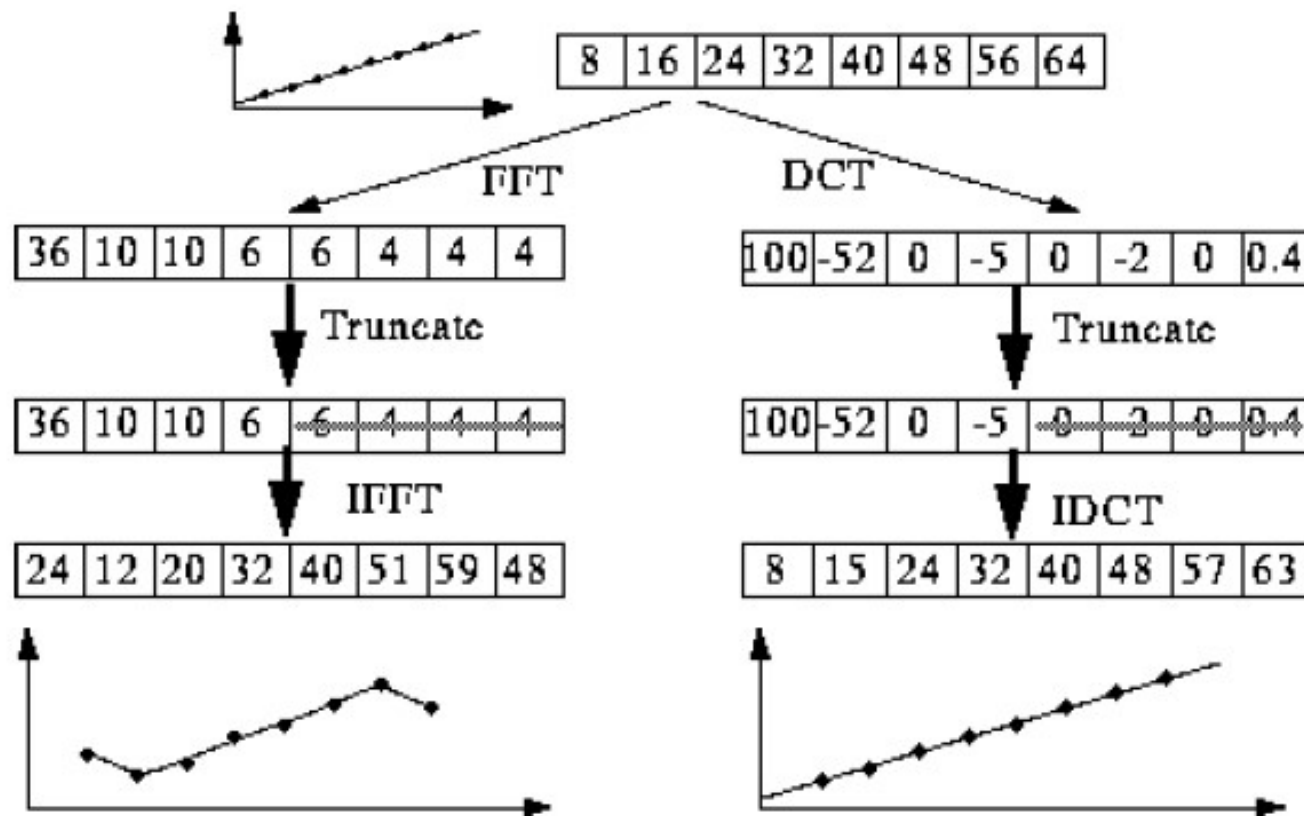
La **Transformée de Fourier est complexe** sans que sa partie réelle ou imaginaire seule ne permette de représenter (donc de reconstruire) l'image.

La DCT est une transformation spectrale qui

- possède les mêmes propriétés que la DFT ;
- est définie par des fonctions de base en *cosinus* seulement ;
- **réelle**
- meilleure **concentration de l'information** que la TFD
- la transformation la plus utilisée en compression (JPEG, MPEG).

Transformée en cosinus discrète DCT

Intérêt de la DCT:

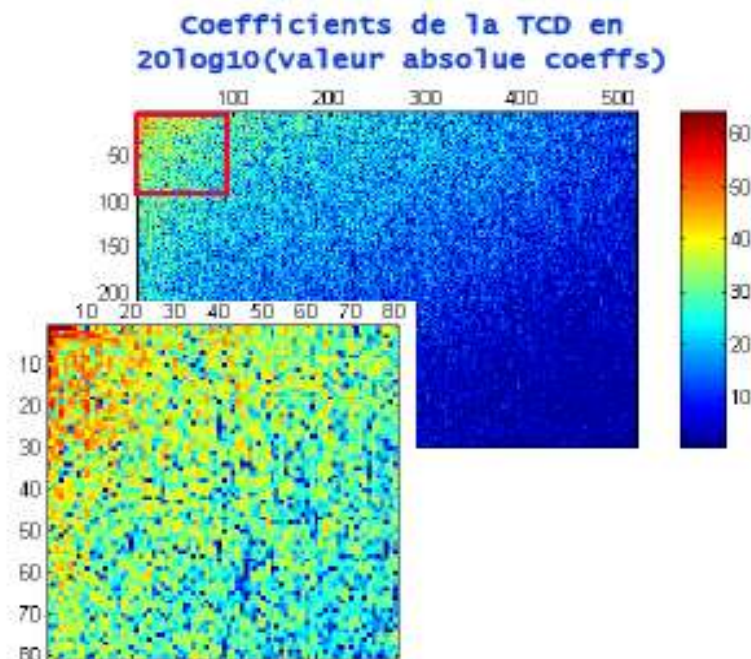


⇒ Un petit nombre de coefficients suffit pour représenter l'image correctement

Transformée en cosinus discrète DCT

Intérêt de la TCD:

- La TCD tend à concentrer toute l'information dans les premiers coefficients. Ces coefficients représentent les **informations importantes** de l'image, et sont liées aux **basses fréquences**.
- Les autres coefficients sont nuls ou quasi-nuls, et correspondent à des plus **hautes fréquences**.



Transformée en cosinus discrète DCT

DCT et DCT inverse:

$$F(u, v) := \frac{2}{\sqrt{MN}} c(u) c(v) \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) \underbrace{\cos\left(\pi \frac{(2m+1)u}{2M}\right)} \underbrace{\cos\left(\pi \frac{(2n+1)v}{2N}\right)}$$

Coef. de normalisation :

$$c(\alpha) := \begin{cases} 1/\sqrt{2} & \text{si } \alpha=0, \\ 1 & \text{si } \alpha \neq 0. \end{cases}$$

pour $\alpha \in [u, v]$

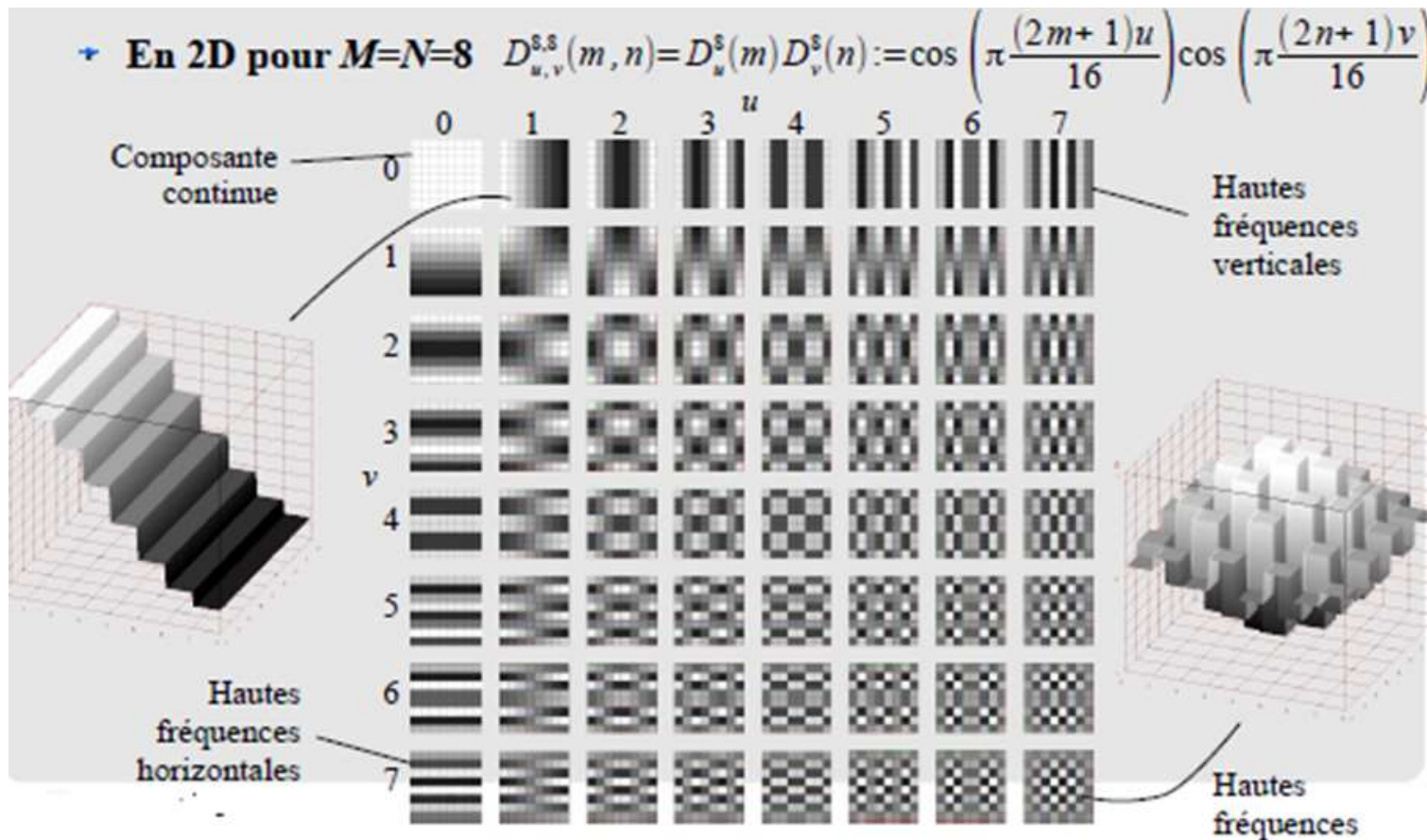
A la matrice initiale f de taille $(N \times M)$ en (m, n) (*espace*), la transformation fait correspondre une matrice F en (u, v) (*fréquences*).

$$f(m, n) := \frac{2}{\sqrt{MN}} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} c(u) c(v) F(u, v) \overbrace{\cos\left(\pi \frac{(2m+1)u}{2M}\right)} \overbrace{\cos\left(\pi \frac{(2n+1)v}{2N}\right)}$$

Transformée en cosinus discrète DCT

Fonctions de base de la Transformée DCT :

- Les fonctions situées en haut et à gauche représentent les basses fréquences
- Les fréquences spatiales augmentent au fur et à mesure que l'on se déplace vers le coin inférieur droit du bloc.



La compression JPEG

La compression JPEG

JPEG : *Joint Photographic Expert Group*

Commission issue de l'Organisation des Standards Internationaux (ISO) chargée de définir **une norme** de format pour la compression d'une image fixe numérique

- Début d'activité~1982
- 1986 : Rejoint par le CCITT
- 1988 : sélection de la DCT (TCD : transformée en Cosinus Discrets)
- 1991-92 : JPEG est un standard international

Depuis cette norme s'est répandue dans tous les domaines du multimédia

La compression JPEG



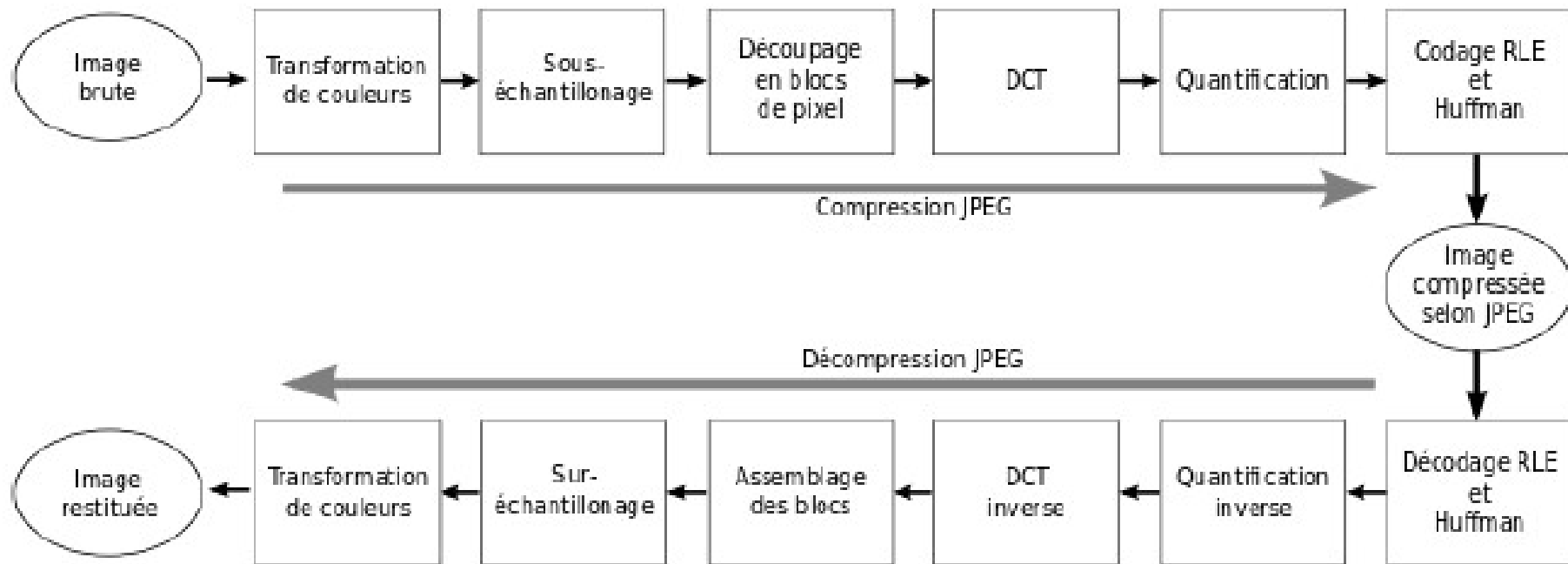
Non compressée
300*225 pixels
200 ko
60s à débit 28 kb/s



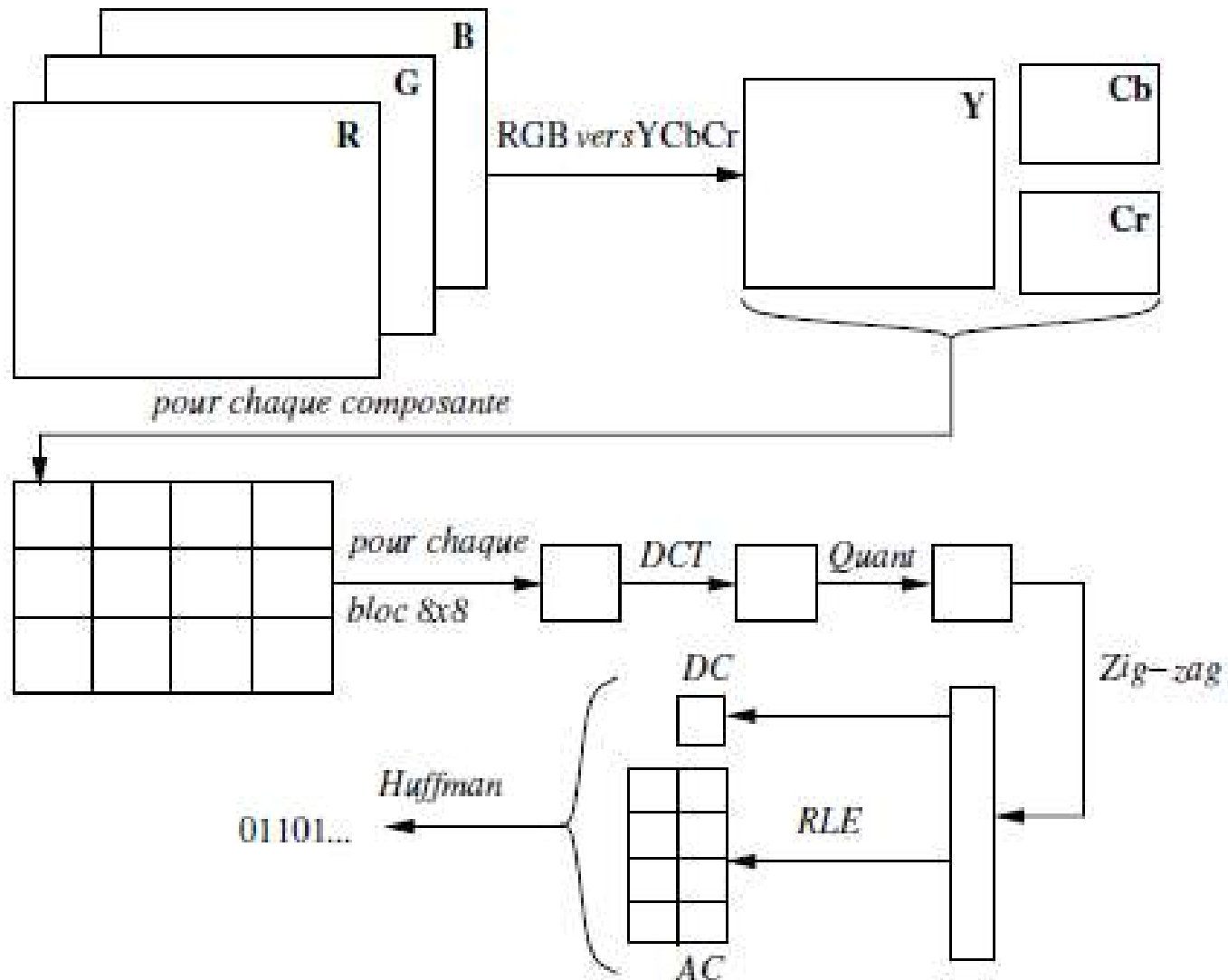
Très fortement compressée en JPEG
300*225 pixels
6 ko
2s à débit 28 kb/s



Principe général



Principe général



Transformation RGB $\rightarrow$ YC_bC_r

- Changement de l'espace couleur RGB $\rightarrow$ YCrCb ou YUV
 - Y composante de luminance
 - Cr composante de chrominance rouge
 - Cb composante de chrominance Bleu



$$\begin{cases} Y &= w_R \cdot R + (1 - w_B - w_R) \cdot G + w_B \cdot B \\ C_b &= \frac{0,5}{1 - w_B} \cdot (B - Y) \\ C_r &= \frac{0,5}{1 - w_R} \cdot (R - Y) \end{cases}$$

$$w_R = 0,299 \quad w_G = 0,587 \quad w_B = 0,114$$

Transformation RGB $\rightarrow$ YC_bC_r

- L'oeil humain est moins sensible aux détails de l'information de couleur (**chrominance**) que de ceux de l'intensité (**luminance**).
 - La représentation RVB n'est pas la plus adaptée
 - L'espace YUV ou YCrCb sépare la luminosité du pixel, de ses chrominances
- Possibilité de **sous-échantillonner** les composantes de **chrominance** sans perte visible de qualité

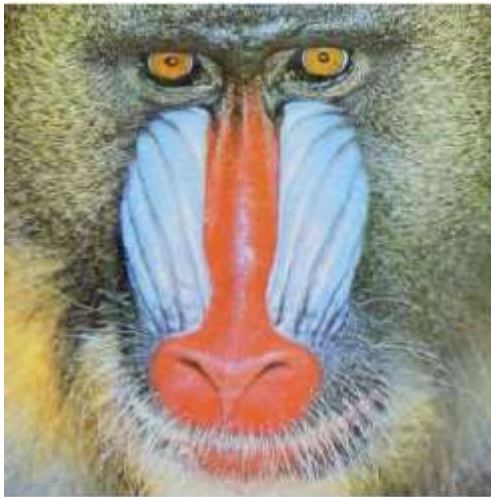
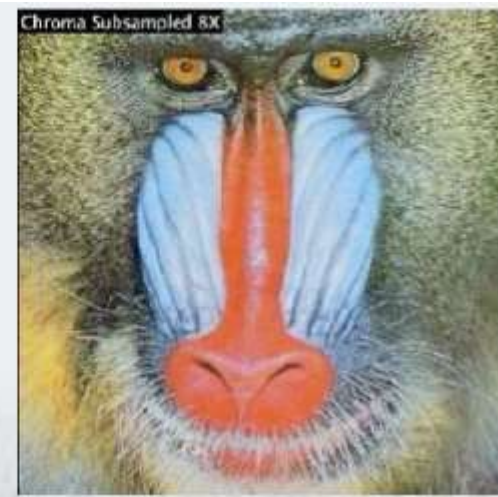


Image originale



Sous échantillonnage de
la luminance

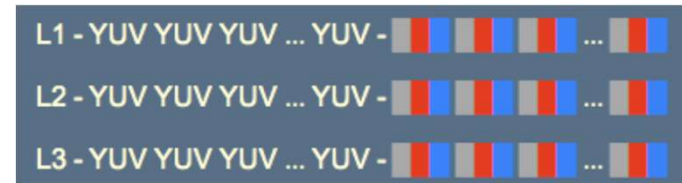


sous échantillonnage des
chrominances

Transformation RGB $\rightarrow$ YC_bC_r

Echantillonnage 4:4:4 (pas de sous-échantillonnage)

- 4 échantillons de luminance (Y)
- 4 échantillons de chrominance rouge (Cr)
- 4 échantillons de chrominance bleue (Cb)



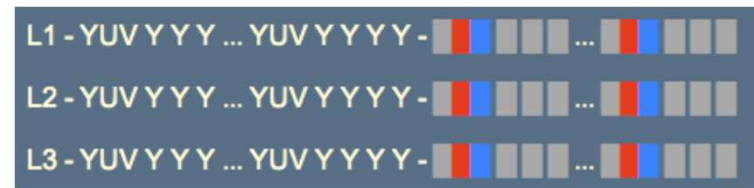
Sous échantillonnage 4:2:2

- 4 échantillons de luminance (Y)
- 2 échantillons de chrominance rouge (Cr)
- 2 échantillons de chrominance bleue (Cb)



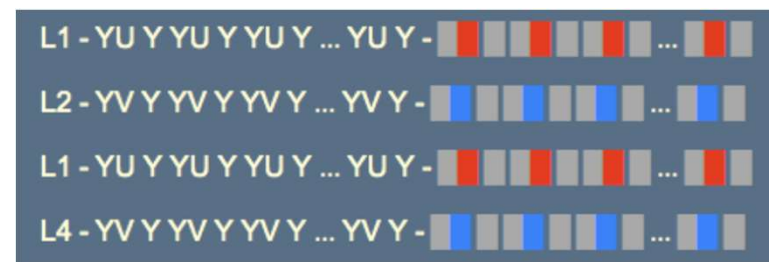
Sous échantillonnage 4:1:1

- 4 échantillons de luminance (Y)
- 1 échantillons de chrominance rouge (Cr)
- 1 échantillons de chrominance bleue (Cb)



Sous échantillonnage 4:2:0

- 4 échantillons de luminance (Y)
 - 2 échantillons (Cr) Ou 2 échantillons (Cb)
- Chaque couleur est codée une ligne sur deux



Découpage par blocs

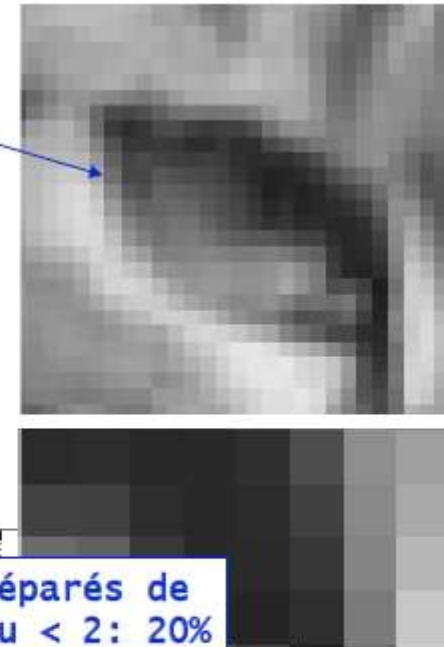
Les images sont localement très redondantes

- Les valeurs des pixels d'une image sont souvent presque égales à celles de leurs voisins, ce qui diminue lorsque l'on considère des pixels de plus en plus éloignés

Pourcentage de pixels voisins dont la différence de niveau est < 2 : 52%

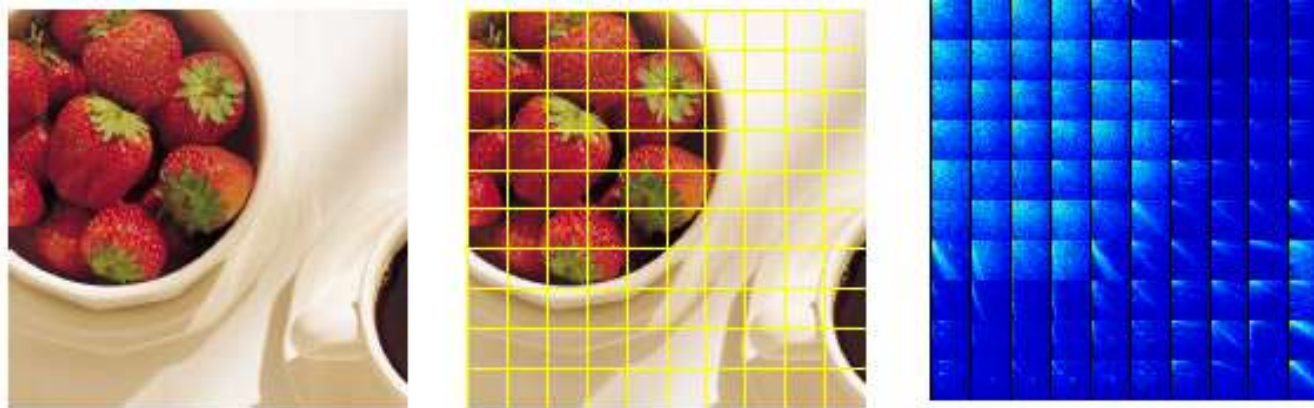


Pourcentage de pixels séparés de 4 pixels avec diff niveau < 2 : 20%



Découpage par blocs

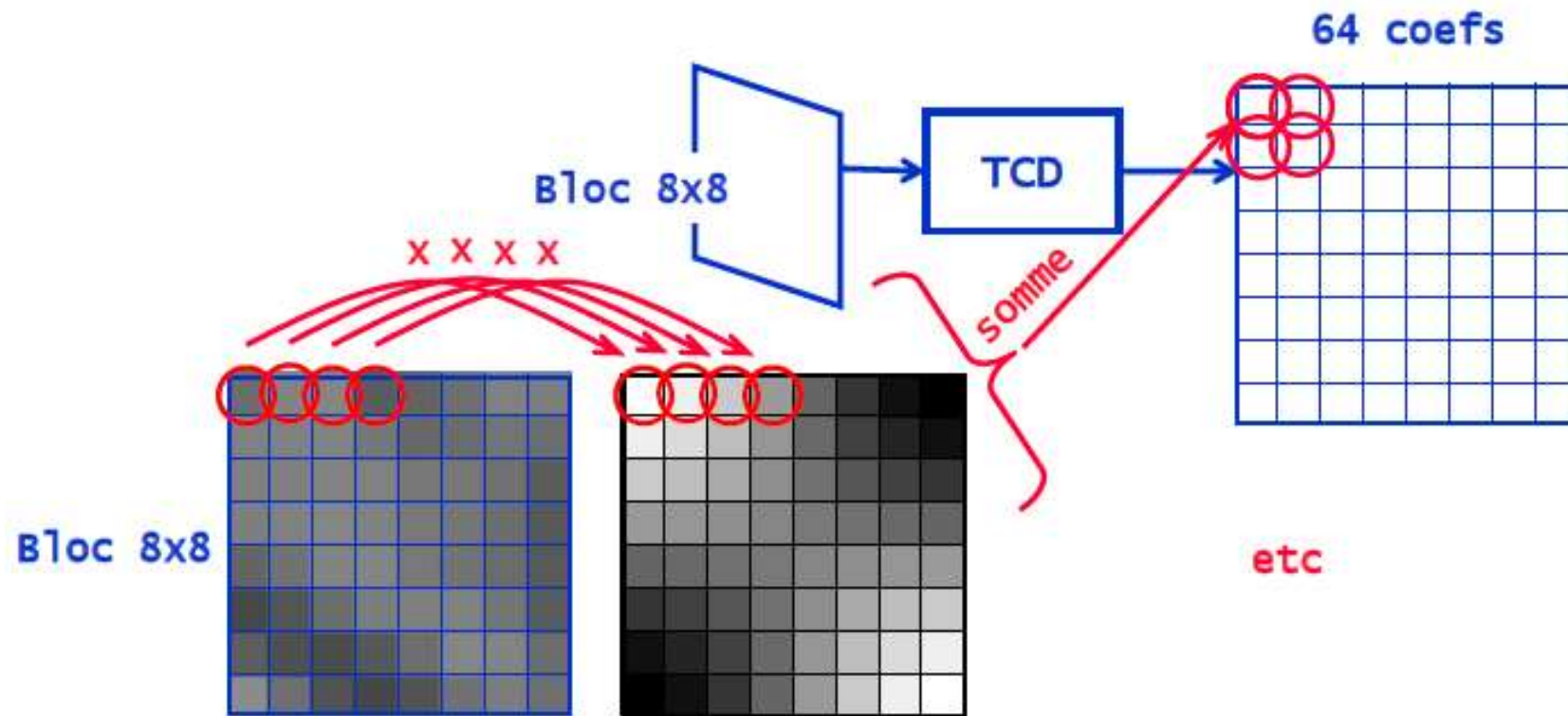
- Découpage de l'image en des blocs de taille $k \times k$ et calcul de la transformée en cosinus discrète de chacun des blocs
- Le choix de la taille du bloc k a une importance capitale :
 - si k est trop faible, le nombre de coefficients est également faible, et ils ont donc tous des valeurs importantes
 - si k est trop grand, alors les valeurs à traiter peuvent devenir trop décorrélées.
- En pratique, une valeur de **$k=8$ est un bon compromis**, et la plupart des algorithmes de compression utilisent une telle valeur



Transformée en cosinus discrète DCT

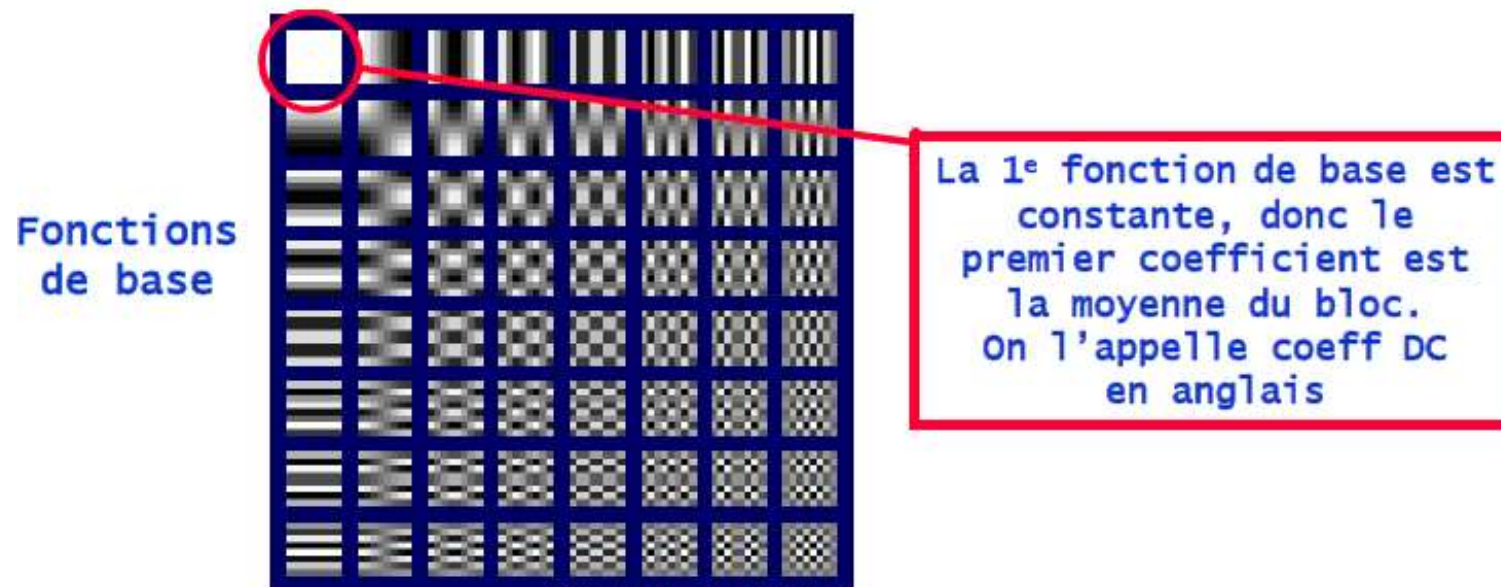
Transformée en cosinus discret de bloc 8x8

- Décomposition du bloc sur 64 fonctions de base 8x8
- Fournit les 64 coefficients de la transformée du bloc
- Calcul du coefficient (u,v) : Multiplication du bloc par la fonction de base (u,v) terme à terme puis somme des 64 valeurs obtenues



Transformée en cosinus discrète DCT

Transformée en cosinus discret de bloc 8x8



Remarques

- Chaque sous-bloc en 8x8 est transformé en une autre matrice par la DCT
- Il n'y a pas de perte d'information ici

Transformée en cosinus discrète DCT

Exemple de bloc 8x8

173	171	171	143	109	100	91	96
171	169	150	137	112	101	94	96
184	158	139	120	110	107	94	100
170	156	134	119	117	104	98	99
157	147	125	127	103	109	90	98
149	146	132	120	113	107	101	93
147	141	119	119	111	101	100	92
160	122	117	116	115	116	102	95

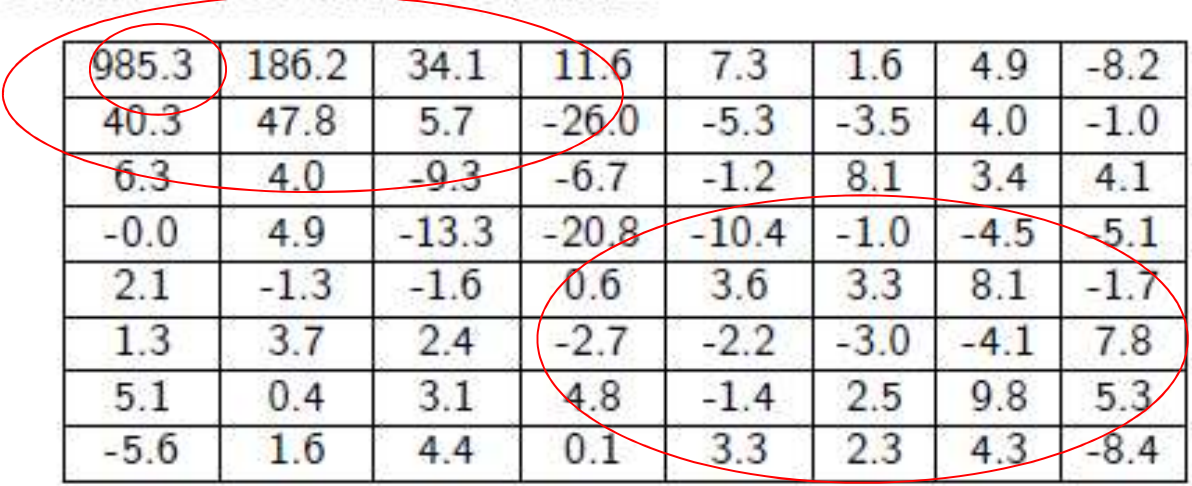
Coefficients TCD du bloc considérée

985.3	186.2	34.1	11.6	7.3	1.6	4.9	-8.2
40.3	47.8	5.7	-26.0	-5.3	-3.5	4.0	-1.0
6.3	4.0	-9.3	-6.7	-1.2	8.1	3.4	4.1
-0.0	4.9	-13.3	-20.8	-10.4	-1.0	-4.5	-5.1
2.1	-1.3	-1.6	0.6	3.6	3.3	8.1	-1.7
1.3	3.7	2.4	-2.7	-2.2	-3.0	-4.1	7.8
5.1	0.4	3.1	4.8	-1.4	2.5	9.8	5.3
-5.6	1.6	4.4	0.1	3.3	2.3	4.3	-8.4

Transformée en cosinus discrète DCT

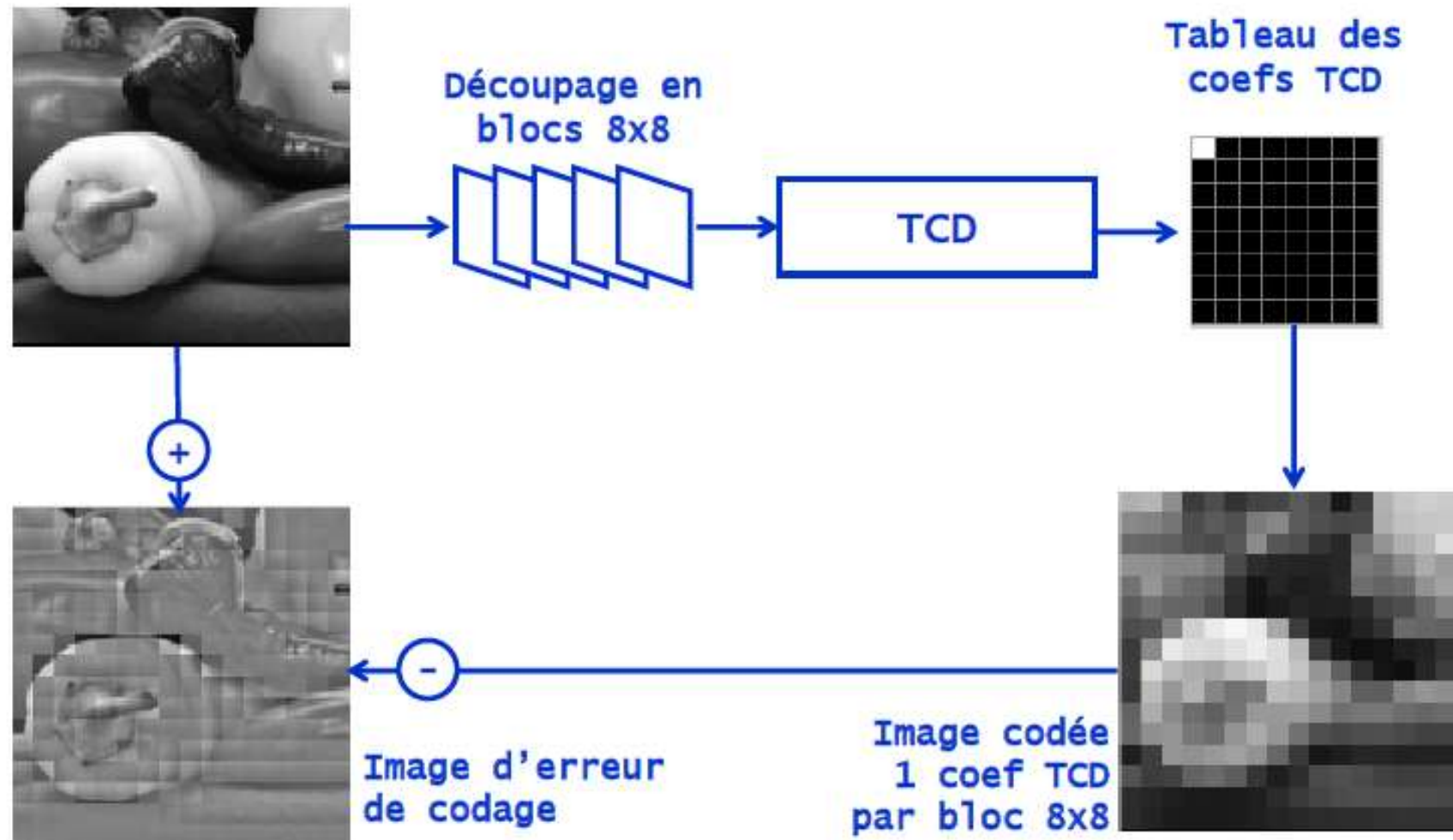
- Composante continue (au coin en haut à gauche)
- Basses fréquences (en haut à gauche)
- Hautes fréquences (en bas à droite)

Coefficients TCD du bloc considérée

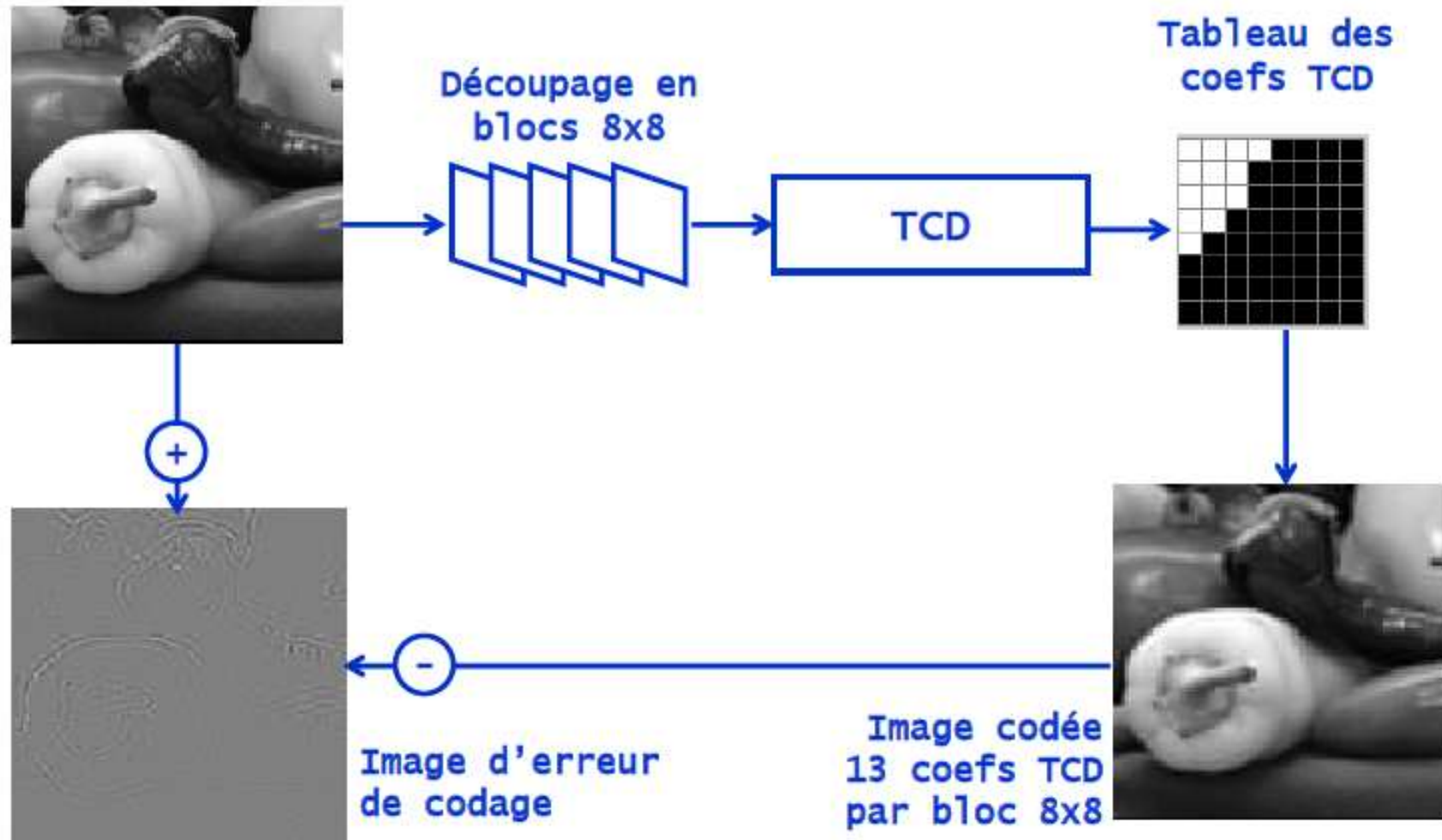


985.3	186.2	34.1	11.6	7.3	1.6	4.9	-8.2
40.3	47.8	5.7	-26.0	-5.3	-3.5	4.0	-1.0
6.3	4.0	-9.3	-6.7	-1.2	8.1	3.4	4.1
-0.0	4.9	-13.3	-20.8	-10.4	-1.0	-4.5	-5.1
2.1	-1.3	-1.6	0.6	3.6	3.3	8.1	-1.7
1.3	3.7	2.4	-2.7	-2.2	-3.0	-4.1	7.8
5.1	0.4	3.1	4.8	-1.4	2.5	9.8	5.3
-5.6	1.6	4.4	0.1	3.3	2.3	4.3	-8.4

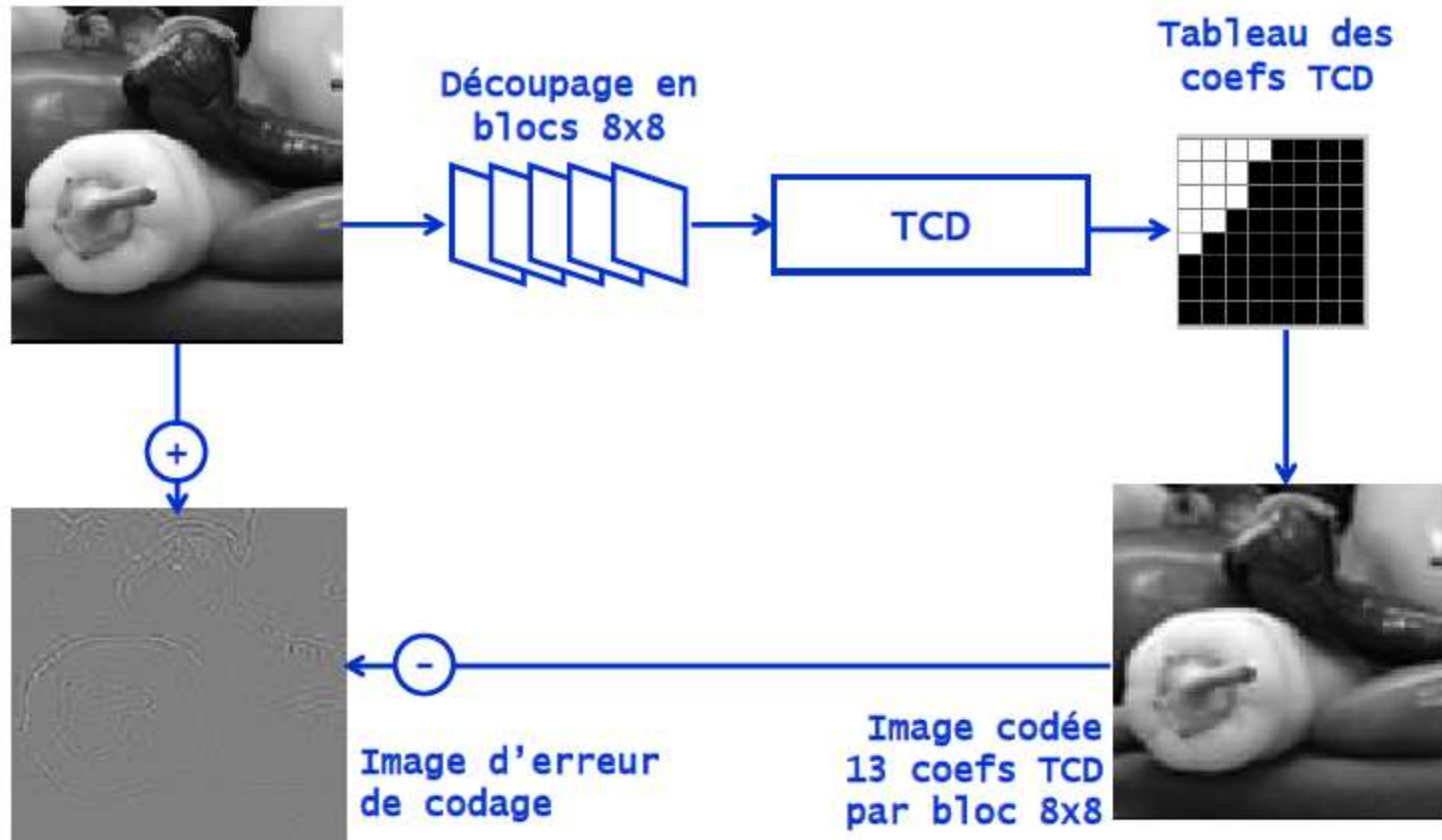
Transformée en cosinus discrète DCT



Transformée en cosinus discrète DCT



Transformée en cosinus discrète DCT



La quantification

Principe

- Réduire la quantité d'information des hautes fréquences.
- L'oeil humain est plus sensible aux basses fréquences.
- **Principale opération destructive** dans JPEG.

Implémentation

Division de chaque coefficient de la matrice DCT par un nombre (quantum), puis arrondi à l'entier le plus proche.

$$F^*[u, v] = \left[\frac{F[u, v]}{Q[u, v]} \right]$$

- Les diviseurs sont donnés dans une **matrice de quantification Q**.
- Le standard JPEG fournit une matrice pour la luminance et pour la chrominance.

La quantification

Exemple

$$f = \begin{bmatrix} 139 & 144 & 149 & 153 & 155 & 155 & 155 & 155 \\ 144 & 151 & 153 & 156 & 159 & 156 & 156 & 156 \\ 150 & 155 & 160 & 163 & 158 & 156 & 156 & 156 \\ 159 & 161 & 162 & 160 & 160 & 159 & 159 & 159 \\ 159 & 160 & 161 & 162 & 162 & 155 & 155 & 155 \\ 161 & 161 & 161 & 161 & 160 & 157 & 157 & 157 \\ 162 & 162 & 161 & 163 & 162 & 157 & 157 & 157 \\ 162 & 162 & 161 & 161 & 163 & 158 & 158 & 158 \end{bmatrix}$$

Bloc 8x8 original

→

$$F = \begin{bmatrix} 1260 & -1 & -12 & -5 & 2 & -2 & -3 & 1 \\ -23 & -17 & -6 & -3 & -3 & 0 & 0 & -1 \\ -11 & -9 & -2 & 2 & 0 & -1 & -1 & 0 \\ -7 & -2 & 0 & 1 & 1 & 0 & 0 & 0 \\ -1 & -1 & 1 & 2 & 0 & -1 & 1 & 1 \\ 2 & 0 & 2 & 0 & -1 & 1 & 1 & -1 \\ -1 & 0 & 0 & -1 & 0 & 2 & 1 & -1 \\ -3 & 2 & -4 & -2 & 2 & 1 & -1 & 0 \end{bmatrix}$$

Bloc 8x8 après TCD

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

Table de quantification

→

$$F^* = \begin{bmatrix} 79 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Bloc 8x8 quantifié 49

La quantification

Quantification des coefficients de la DCT

JPEG définit des tables de quantification par défaut, dont les valeurs sont le résultat de nombreuses mesures réalisées par le comité.

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	36	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Quantification de la luminance

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

Quantification de la chrominance

La quantification

Quantification des coefficients de la DCT

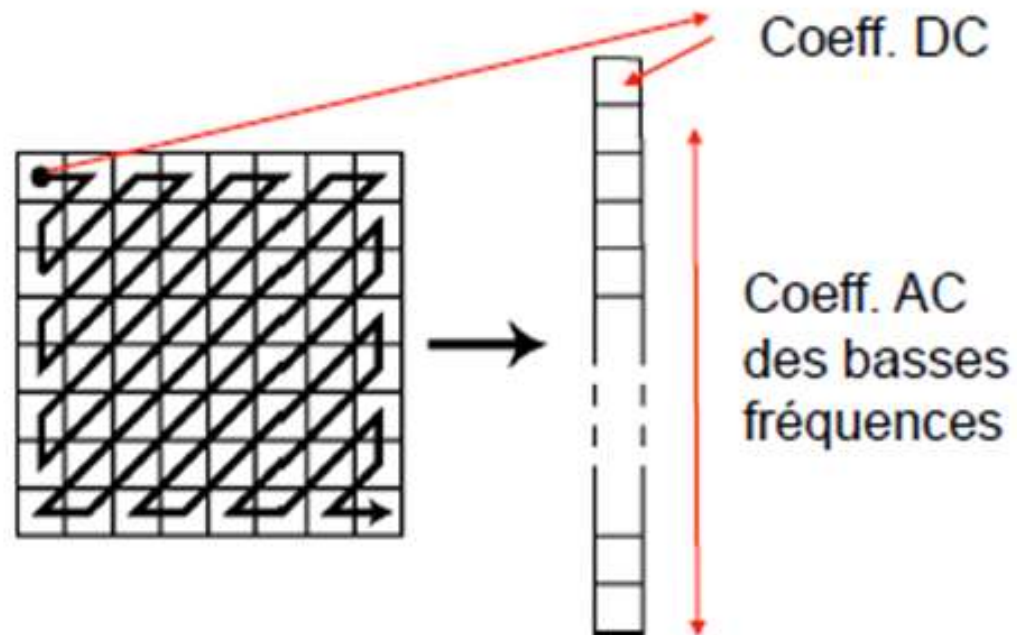
Il est également possible de définir sa propre table de quantification, par exemple avec :

$$Q(i, j) = 1 + (i + j)R$$

1	3	5	7	9	11	13	15
3	5	7	9	11	13	15	17
5	7	9	11	13	15	17	19
7	9	11	13	15	17	19	21
9	11	13	15	17	19	21	23
11	13	15	17	19	21	23	25
13	15	17	19	21	23	25	27
15	17	19	21	23	25	27	29

$$R=2$$

Codage entropique



- Coefficient DC : codage prédictif + Huffman
- Coefficients AC : codage "run-length" + Huffman

Codage des coefficients DC

Coefficient DC : 1er coefficient = moyenne du bloc

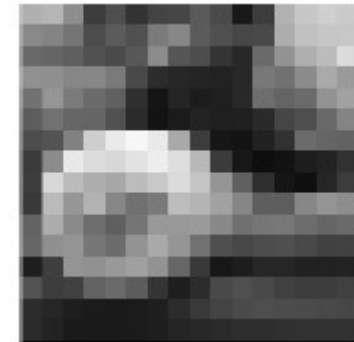
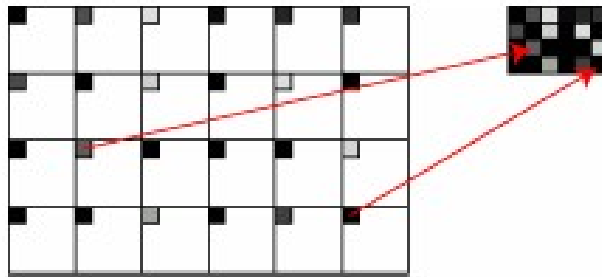
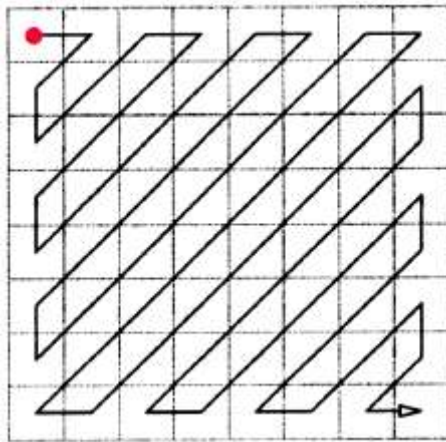
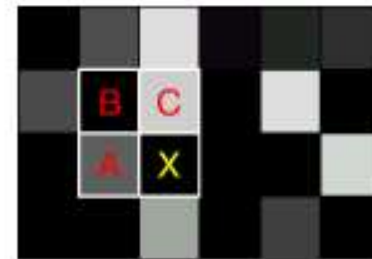
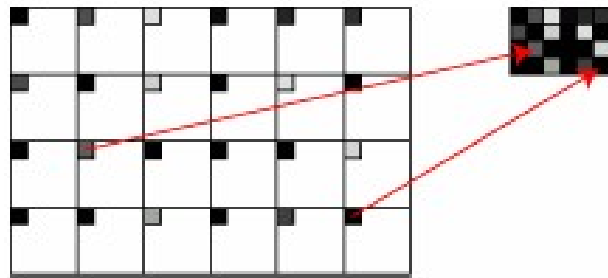
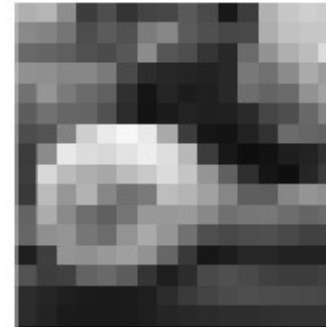


Image des coefficients DC :
valeurs voisines souvent proches

Codage des coefficients DC

Méthode:

- Les coefficients (DC) de chaque bloc i , qui concentrent la majeure partie de l'énergie, varient peu d'un bloc à l'autre.
- Regroupement des coefficients DC (balayage de gauche à droite, et de haut en bas)



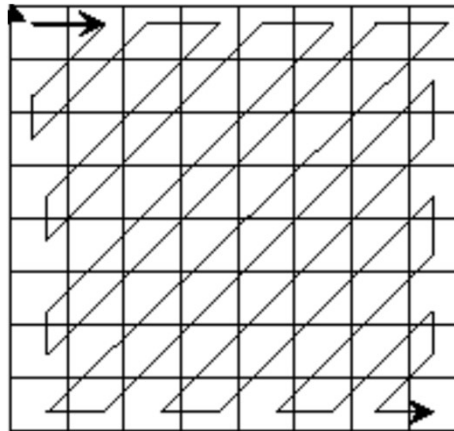
- La séquence $\{DC^i\}_{0 \leq i < P}$ est codée par **codage différentiel** (prédicatif)
On code la différence entre le coef DC du bloc et le coef DC du bloc précédent

DC^0	$DC^1 - DC^0$	$DC^2 - DC^1$	...	$DC^{P-2} - DC^{P-1}$
--------	---------------	---------------	-----	-----------------------

Codage des coefficients AC

Les autres coefficients (AC) de chaque bloc i sont lus en zigzag.

Balayage en Zig-Zag



Intérêt:

- former un vecteur où les coefficients relatifs aux basses fréquences sont regroupés
- concentrer les coefficients nuls à la fin du balayage

Codage des coefficients AC

Constat : apparition de longues plages de 0 après quantification

Méthode :

1. codage de ces plages («Run Length Coding»)
 - un ensemble de paires (**Nb. de 0** , **Coeff.**)
 - fin d'un bloc : paire (0, 0) :
2. Codage de Huffman des paires (**Nb. de 0** , **Coeff.**)

Coefficients quantifiés et codés

61	16	3	0	0	0	0	0
3	3	0	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

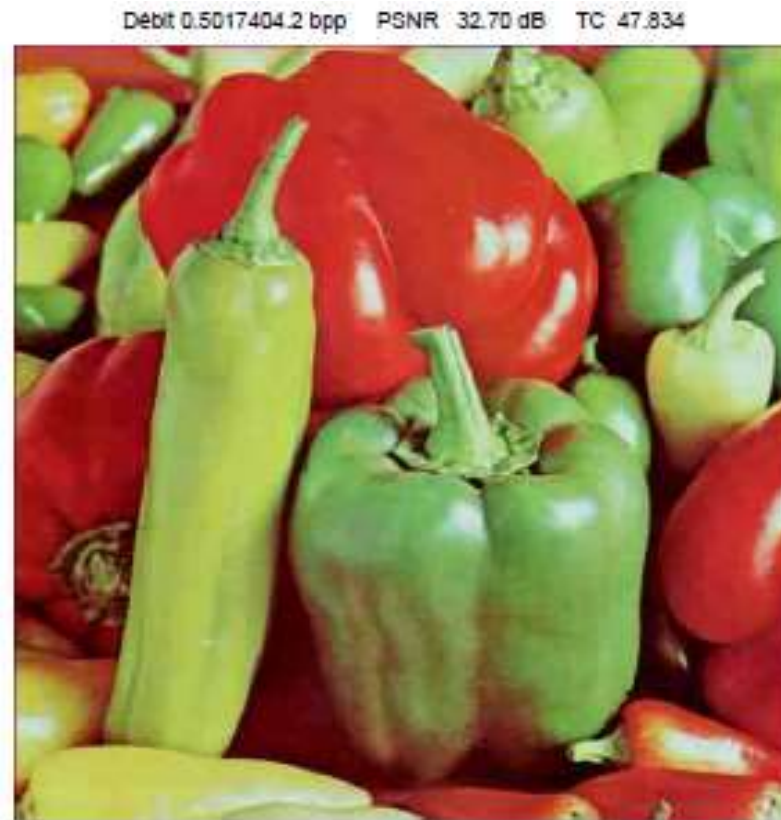
$61-dc_{k-1}$	0,16	0,3	1,3	0,3	7,-1	EOB
---------------	------	-----	-----	-----	------	-----

Performances de la compression JPEG

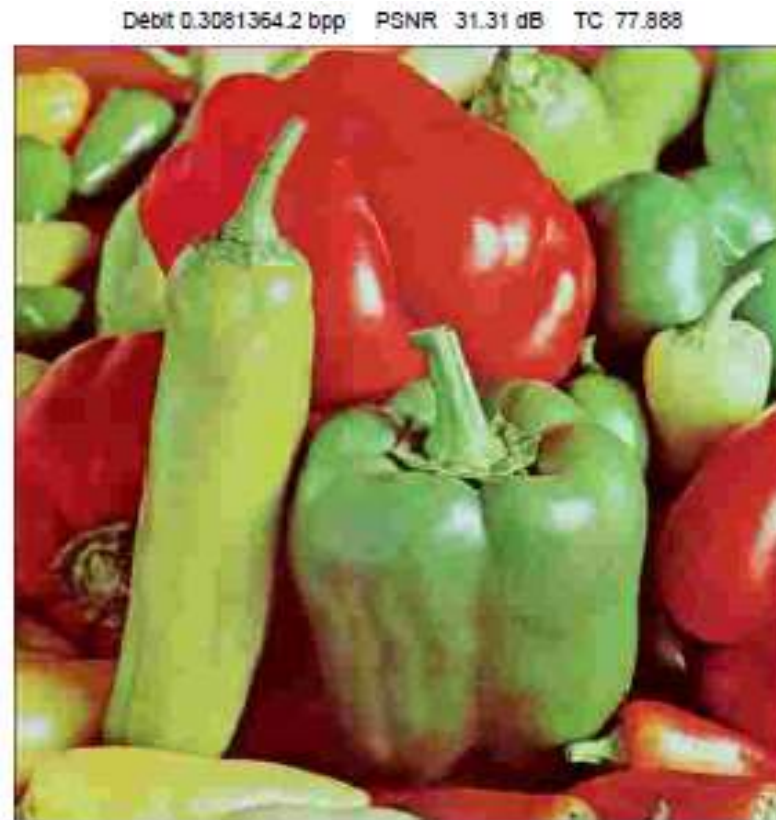
Image Originale, 24 bpp



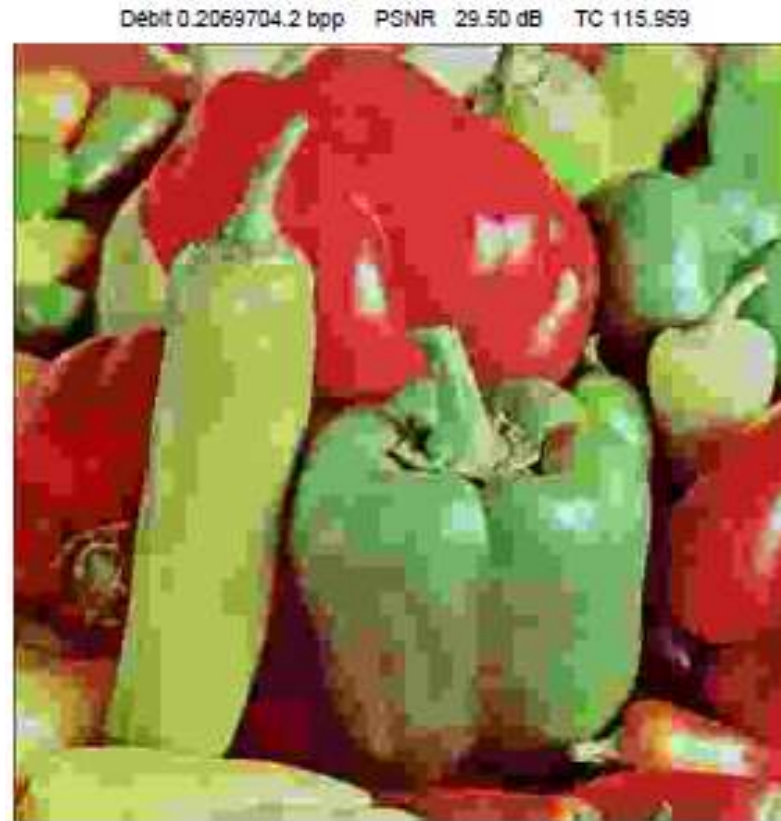
Performances de la compression JPEG



Performances de la compression JPEG



Performances de la compression JPEG



- **Apparition de blocs** d'autant plus visibles que le taux de compression augmente.
- Baisse de la qualité de l'image